

Side Effect Term Matching using Graph Mining

The two sets of side effect names that need to be matched are follows

1. SIDER has 6120 side effect names
2. 192 side effect name achieved from pharmaceutical profiling

For each of the side effect names from two sides there are around 100 documents collected from MEDLINE data source. MEDLINE is an unstructured data source for medical publications. It contains almost 30 million citations of journal. For each journal MEDLINE has the abstract and title name. We get the data stored them in a Jason format.

For each of the side effect name we took the all the document associated with it and generated the bigrams. Bigrams are used to retrieve all the associated concept unique identifier(CUI) associated with the bigram. In this way we collected a set of CUIs associated with each of the document. So for each bigram there will be multiple concepts in order to make these concepts useful for our process we need to find out the best fit CUI for each bigram that we did in the next section.

We generated the following graph:

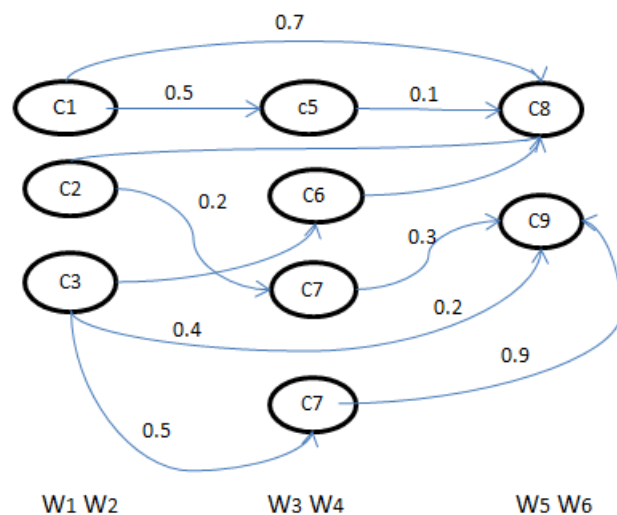


Fig 1: graph for bigrams associated with concepts

So the each of the bigrams generated from the documents are associated with multiple concept unique identifier. The goal of the above graph is to find out the best associated CUI for each of the bigram. Each column of CUIs of the above graph is associated with one bigram. The edge of the graph between any two CUIs is created if there is any association between them based on some concept model measurement. Concept model has been created using the technique of language model. The goal of a

language model is to predict the correct missing word for a given sentence in order to complete the sentence. Using this technique we can learn word vectors. Word vectors are multi dimensional numerical presentation of words which captures the syntactic and semantic relationship hidden in the text. We can apply the same technique to generate the concept vectors. CUIs are the high level abstraction for a group of words. Each CUI represent a set of words which has similar semantic meaning and represents similar semantic relationship in the sentence. Once we represent the words in Eucledian space using vectors then we can perform the linear operation on them. Such as if we take word vector king and subtract it from word vector man then add word vector woman the resultant word vector is very close to queen as follows,

$$\text{vector}(\text{"king"}) - \text{vector}(\text{"man"}) + \text{vector}(\text{"woman"}) \approx \text{vector}(\text{"queen"})$$

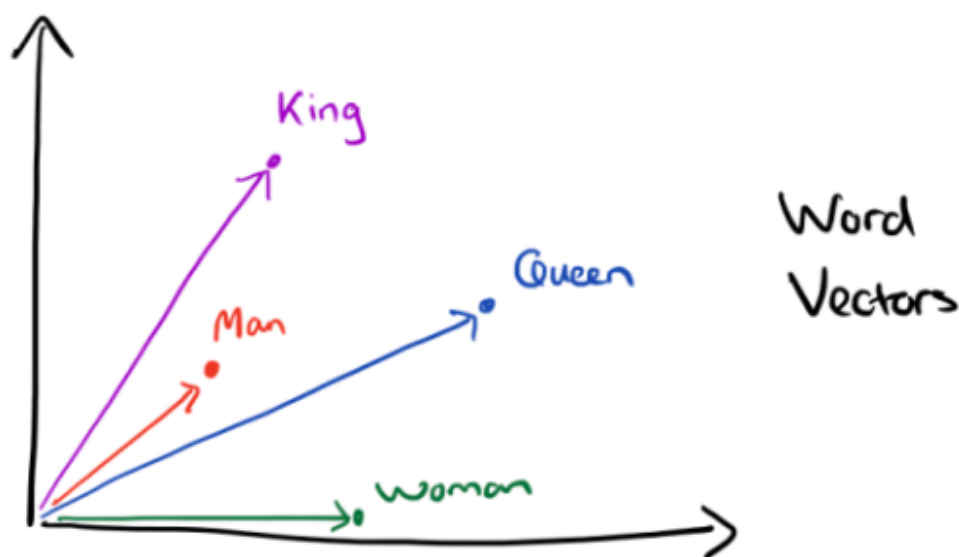


Fig 2: resultant queen vector

The technique to generate low dimensional word vectors is to use following simplified neural network models. In this model we pick one sentence from our training set fed the first few words of the sentence in the input layer and try to predict the last few words in the output layer. The goal is to maximize the output layer softmax function while we try to minimize the cost generated by the model. This cost minimization process helps the model to optimize the parameters associated with the model and as a part of the parameter set of the model the word vectors also get optimized. When the training process completes, we get this nice learned word vectors that capture the semantic and syntactic information hidden in the text. We can simply apply linear operations such as cosine similarity, vector addition, projection on the word vectors to achieve nice measurements that can be used in decision making process. Word vectors can be look like as follows,

King	0.01	0.04	0.001	0.7	0.0071	0.0101	0.230	0.1211	0.11
------	------	------	-------	-----	--------	--------	-------	--------	------

Man	0.01	0.08	0.1	0.7	0.0071	0.0101	0.230	0.1211	0.11
-----	------	------	-----	-----	--------	--------	-------	--------	------

So the word is numerically represented by each dimension of the word vector and multiple word vectors may have similar vector component representing similarity between two word vectors. That means if two words are semantically similar they will have similar word vector component shared between them.

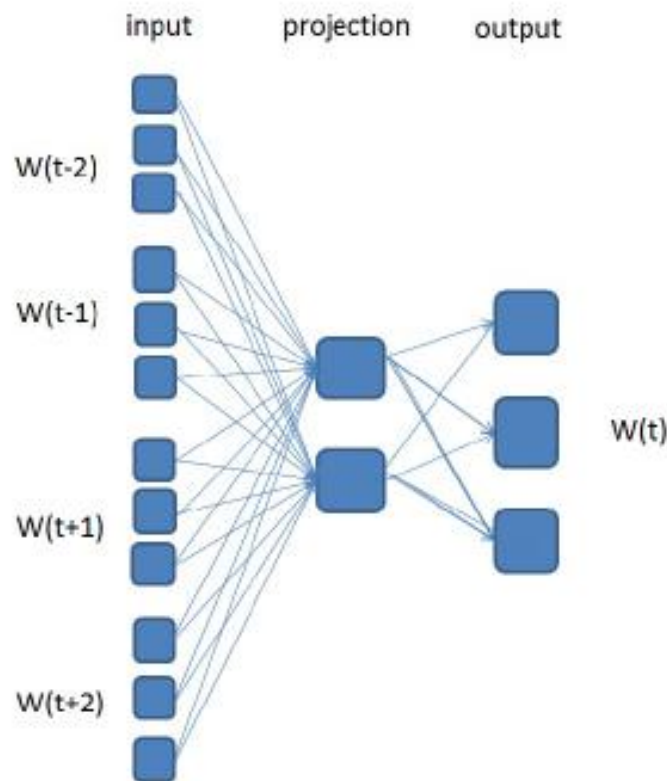


Fig: Bag of word neural model

The neural model has three layers. The first is input layer, randomly initialized word vectors has to be presented in the input layer. Second is the projection layer that project the input vectors into low dimensional projection layer and this player shared by all the word vectors. Final layer is the softmax output layer. Here in the above model we presented previous two words and next two words of a current word of interest as an input to the model and the model tried predict the current word of interest in the output layer.

As it is described earlier the entire MEDLINE data is used to get the input sentences to train this model. Same model can be used to get learned vectors for concepts. Just by replacing each word of the MEDLINE text with its concept retrieved from metamap we can generate a huge document of CUIS. Where CUIS are placed next to each other based on the textual context. Now it is possible to use the CUI documents to generate the CUI vectors using the same above model.

Once the CUI vectors are generated the cosine similarity measurement is used to generate the similarity score between two CUIs and if it is greater than a minimum required value then an edge between two CUIs is created in the graph and the similarity score is assigned as a weight of the edge. Once the graph is built shown on above Fig 1, then graph based centrality algorithm is used to measure the weight for each node. The algorithm that we used here is the weighted page rank.

Weighted Page Rank

The page rank algorithm works by voting or recommendation. Each node votes to its neighboring connected node. Node with higher number of votes is more important besides the importance of the node casting the vote is also taken into account and determines the how important the vote it is casting. In a weighted page rank the weight of the outgoing edge is also considered with a higher likelihood of following an edge with a larger weight.

$$PageRank(V_b) = (1 - d) + d \sum_{(V_a, V_b) \in E} \frac{w_{a,b}}{\sum_{(V_c, V_a) \in E} w_{c,a}} PageRank(V_a)$$

Where V_a is the current node of interest and V_b is the end node of one of the out going edge from node V_a .

After running the weighted page rank on the above graph we end up getting a weight score for each of the node. We assigned the bigram to that CUI node which has the highest score. In this way, we will get filtered best CUI for each of the bigram. This CUIs are the most relevant CUIs to the words based on the context they are used.

Evaluation Procedure for measuring similarity

Once we get the cui sets for SIDER side effects and the target side effect names, it is possible to determine the similarity between any two side effect names from this two sets. The technique we used is as follows,

- We took a CUI pair from target side effect names and SIDER side effect names, then using the co-occurrence frequency measurement from UMLS concept network we can get one measurement. UMLS concept network contains the cooccurrence relationship between any two concepts. Mathematically it can be described as follows,

$$f(C_1, C_2) = \frac{Count(r(C_1, C_2))}{Count(., C_2)}$$

Where C_1 belongs to SIDER side effect name concepts and C_2 is related to target side effect name.

- Second Measurement is the cosine similarity based measurement that is $CosineSimilarity(C_1, C_2)$

Max generated score SIDER side effect name is our candidate which has a semantic similarity with our target side effect names.